

Updating a Cached QR Factorization with Givens Rotations

Consider a full-rank matrix $A \in \mathbb{R}^{m \times n}$ with QR factorization

$$A = QR, \quad R = \begin{pmatrix} r_{11} & r_{12} & \cdots & r_{1n} \\ & r_{22} & \cdots & r_{2n} \\ & & \ddots & \vdots \\ & & & r_{nn} \end{pmatrix},$$

and cached right-hand side $c = Q^T b$. Appending a single constraint $a^T x = c_{n+1}$ produces the augmented system

$$\begin{pmatrix} A \\ a^T \end{pmatrix} x' = \begin{pmatrix} b \\ c_{n+1} \end{pmatrix}.$$

Multiplying by Q^T keeps the orthogonal factor implicit:

$$\begin{pmatrix} R \\ w^T \end{pmatrix} x' = \begin{pmatrix} c \\ c_{n+1} \end{pmatrix}, \quad w^T = a^T Q.$$

Only the last row breaks triangular form, so we repair it with Givens rotations instead of recomputing the factorization from scratch.

Which rows do we rotate?

Each rotation acts on a 2×2 block of rows. To eliminate the entry below the diagonal in column j , we rotate

$$\text{extrow } j \text{ and row } n+1,$$

leaving every other row untouched. The rotation matrix is

$$G_{j,n+1}(c, s) = \begin{pmatrix} I_{j-1} & & & \\ & c & & s \\ & & I_{n-j} & \\ & -s & & c \end{pmatrix},$$

with parameters chosen so that

$$\begin{pmatrix} c & s \\ -s & c \end{pmatrix} \begin{pmatrix} r_{jj} \\ w_j \end{pmatrix} = \begin{pmatrix} \hat{r}_{jj} \\ 0 \end{pmatrix}.$$

Because Q changes implicitly, we apply the same rotation to the cached right-hand side:

$$c \leftarrow G_{j,n+1}^T c.$$

Column-by-column sweep

We work from the last column toward the first:

1. For $j = n, n - 1, \dots, 1$:

- (a) Read the pivot r_{jj} from row j and the subdiagonal entry w_j from row $n + 1$.
- (b) Compute the stable rotation

$$r = \sqrt{r_{jj}^2 + w_j^2}, \quad c = \frac{r_{jj}}{r}, \quad s = \frac{w_j}{r}.$$

- (c) Apply $G_{j,n+1}(c, s)$ to rows j and $n + 1$ of R (only columns $j, j + 1, \dots, n$ are touched).
- (d) Rotate the matching entries of w and the cached vector c .

After column j is processed, entry $(n + 1, j)$ becomes zero. Since each step touches just two rows, the overall update costs $O(n)$.

Final triangular form

When the sweep completes we obtain

$$\tilde{R} = \begin{pmatrix} r'_{11} & r'_{12} & \cdots & r'_{1n} \\ & r'_{22} & \cdots & r'_{2n} \\ & & \ddots & \vdots \\ & & & r'_{nn} \\ & & & & r_{n+1,n+1} \end{pmatrix}, \quad \tilde{c} = \begin{pmatrix} c'_1 \\ \vdots \\ c'_n \\ c_{n+1} \end{pmatrix}.$$

Back-substitution on the first n rows gives the updated solution x' , while the final entry tracks the residual of the new constraint.

Two-by-two picture

Each step is just the classic 2×2 rotation

$$\begin{pmatrix} \cos \theta & \sin \theta \\ -\sin \theta & \cos \theta \end{pmatrix} \begin{pmatrix} a \\ b \end{pmatrix} = \begin{pmatrix} \sqrt{a^2 + b^2} \\ 0 \end{pmatrix},$$

applied to rows j and $n + 1$. Recognizing that only those two rows move makes the update workflow easy to visualize and implement.